Exercises

Crypto

Question

<u>You are having a discussion with a friend about cryptography. Your friend makes a series of statements. Please tell us how you would respond (True or False) and motivate your answer.</u>

A. The reason why the 2048bit RSA is more robust to brute forcing that a 256bit AES, is because the key is longer.

B. No encryption algorithm is perfect, as they are all vulnerable to brute forcing.

C. An encryption algorithm is broken when there is at least one way to derive the key from a given amount of ciphertext.

A. The reason why the 2048-bit RSA is more robust to brute forcing that a 256-bit AES, is because the key is longer.

A. The reason why the 2048-bit RSA is more robust to brute forcing that a 256-bit AES, is because the key is longer.

[see slides on "cryptography"] *False. The size of the key cannot be used as a direct comparison criterion because the required computational effort depends on the key length in a different way depending on the cipher.*

B. No encryption algorithm is perfect, as they are all vulnerable to brute forcing.

A. The reason why the 2048-bit RSA is more robust to brute forcing that a 256-bit AES, is because the key is longer.

[see slides on "cryptography"] *False. The size of the key cannot be used as a* <u>*direct*</u> *comparison criterion because the required computational effort depends on the key length in a different way depending on the cipher.*

B. No encryption algorithm is perfect, as they are all vulnerable to brute forcing.

[see slides on "cryptography"] *False. A perfect cipher, i.e., a one-time pad with a key chosen uniformly at random is not vulnerable to an exhaustive search as all the keys will yield a potentially meaningful plaintext (with the same likelihood of being the good one).*

C. An encryption algorithm is broken when there is at least one way to derive the key from a given amount of ciphertext.

A. The reason why the 2048-bit RSA is more robust to brute forcing that a 256-bit AES, is because the key is longer.

[see slides on "cryptography"] *False. The size of the key cannot be used as a* <u>*direct*</u> *comparison criterion because the required computational effort depends on the key length in a different way depending on the cipher.*

B. No encryption algorithm is perfect, as they are all vulnerable to brute forcing.

[see slides on "cryptography"] *False. A perfect cipher, i.e., a one-time pad with a key chosen uniformly at random is not vulnerable to an exhaustive search as all the keys will yield a potentially meaningful plaintext (with the same likelihood of being the good one).*

C. An encryption algorithm is broken when there is at least one way to derive the key from a given amount of ciphertext.

[see slides on "cryptography"] *False. Generally, a cryptosystem is broken if there is any attack faster than trying all the possible keys to obtain the plaintext.*

1.7. You (Eve) have intercepted two ciphertexts:

$$c_1 = \texttt{111110010111100111001100000101111000011 0}$$

$$c_2 = \texttt{11111010011001111101110100001001100010 00}$$

You know that both are OTP ciphertexts, encrypted with the *same key*. You know that **either** c_1 is an encryption of `alpha` and c_2 is an encryption of `bravo` **or** c_1 is an encryption of `delta` and c_2 is an encryption of `gamma` (all converted to binary from ASCII in the standard way).

Which of these two possibilities is correct, and why? What was the key k?

Observe that c_1 = p_1 ^ k and c_2 = p_2 ^ k. Adding memberwise the two equations will yield c_1 ^ c_2 = p_1^p_2. Note that c_1^c_2 is known to the attacker, and it is a bit string where a set bit in a given position tells the attacker that the bits in the corresponding positions of the plaintexts are different. Observe that the value of the xor of the last 8 bits of c_1 and c_2 is not all zeroes, therefore the last characters of the two plaintexts differ. We thus obtain that it's alpha and bravo

Claim 1.3 *For every $m \in \{0,1\}^\lambda$, the distribution* EAVESDROP(m) *is the* ***uniform distribution*** *on $\{0,1\}^\lambda$. Hence, for all $m, m' \in \{0,1\}^\lambda$, the distributions* EAVESDROP(m) *and* EAVESDROP(m') *are identical.*

1.9. Suppose we modify the subroutine discussed in Claim 1.3 so that it also returns k:

$$\boxed{\begin{array}{l} \underline{\textsc{eavesdrop}'(m \in \{0,1\}^\lambda):} \\ \quad k \leftarrow \{0,1\}^\lambda \\ \quad c := k \oplus m \\ \quad \text{return } (k, c) \end{array}}.$$

Is it still true that for every m, the output of EAVESDROP$'(m)$ is distributed uniformly in $(\{0,1\}^\lambda)^2$? Or is the output distribution different for different choice of m?

It's false. Simply consider the value obtained xor-ing the two returned values: it's k^c = m. It's clear that the distribution of m does not look like a random independent bitstring with respect to m, indeed, it's the message!

2.4. Show that the following libraries are **not** interchangeable. Describe an explicit distinguishing calling program, and compute its output probabilities when linked to both libraries:

$\mathcal{L}_{\text{left}}$
$\textsc{eavesdrop}(m_L, m_R \in \{0,1\}^\lambda)$:
$k \leftarrow \{0,1\}^\lambda$
$c := k \oplus m_L$
return (k, c)

$\mathcal{L}_{\text{right}}$
$\textsc{eavesdrop}(m_L, m_R \in \{0,1\}^\lambda)$:
$k \leftarrow \{0,1\}^\lambda$
$c := k \oplus m_R$
return (k, c)

```
tmp_k,tmp_c = eavesdrop(0^lambda,1^lambda)
if (tmp_k^tmp_c = 0^lambda)
   return "it's L_left!"
else
   return "it's L_right!"
```

The distinguisher works with Pr = 1

5.12. Let G_1 and G_2 be deterministic functions, each accepting inputs of length λ and producing outputs of length 3λ.

(a) Define the function $H(s_1 \| s_2) = G_1(s_1) \oplus G_2(s_2)$. Prove that if **either** of G_1 or G_2 (or both) is a secure PRG, then so is H.

(b) What can you say about the simpler construction $H(s) = G_1(s) \oplus G_2(s)$, when one of G_1, G_2 is a secure PRG?

a) Assume that G_1 is the good PRG (the same reasoning applies if it's G_2). If G_1 the output of G_1 cannot be told apart from a uniformly random sampled string of 3*lambda bits, then the value G_1(s_1)^G_2(s_2) is not distinguishable from random by Shannon's theorem (it's essentially a one time pad). As a consequence, the output of H is indistinguishable from a uniformly randomly chosen string.

b) You can reduce this case to the previous one. It's just feeding the previous H function with a non uniformly sampled string.

5.12. Let G_1 and G_2 be deterministic functions, each accepting inputs of length λ and producing outputs of length 3λ.

(a) Define the function $H(s_1 \| s_2) = G_1(s_1) \oplus G_2(s_2)$. Prove that if **either** of G_1 or G_2 (or both) is a secure PRG, then so is H.

(b) What can you say about the simpler construction $H(s) = G_1(s) \oplus G_2(s)$, when one of G_1, G_2 is a secure PRG?

Definition

A CSPRNG is a deterministic function $\text{PRNG}: \{0,1\}^\lambda \to \{0,1\}^{\lambda+l}$ whose output cannot be distinguished from an uniform random sampling of $\{0,1\}^{\lambda+l}$ in $\mathcal{O}(\text{poly}(\lambda))$. l is the CSPRNG stretch.

HINTS

$\text{VERNAM}(\lambda, \texttt{ptx} \in \{0,1\}^\lambda)$

$k \xleftarrow{\$} \{0,1\}^\lambda$

$\texttt{ctx} = \texttt{ptx} \oplus k$

return $\texttt{ctx}$

$\text{IDEAL}(\lambda, \texttt{ptx} \in \{0,1\}^\lambda)$

$\texttt{ctx} \xleftarrow{\$} \{0,1\}^\lambda$

return $\texttt{ctx}$

11.13. Let H be a collision-resistant hash function with output length n. Let H^* denote iterating H in a manner similar to CBC-MAC:

$H^*(x_1 \cdots x_\ell)$:

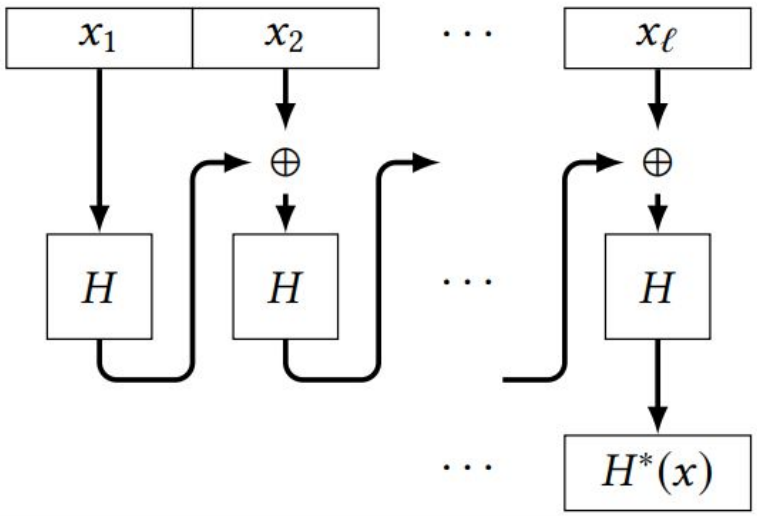
// *each* x_i *is* n *bits*

$y_0 := 0^n$

for $i = 1$ to ℓ:

$\quad y_i := H(x_i \oplus y_{i-1})$

return y_i



Show that H^* is **not** collision-resistant. Describe a successful attack.

Take a random message x=(x_1,..,x_l), compute the output of H*, call it d. Compute pad=d^x_l. H*(x||pad) will have the same hash as x since the last step will compute the hash of pad^H*(x) = d^x_l^H*(x) = x_l

10.1. Consider the following MAC scheme, where F is a secure PRF with $in = out = \lambda$:

KeyGen:	$\mathrm{MAC}(k, m_1 \| \cdots \| m_\ell)$: // *each* m_i *is* λ *bits*
$k \leftarrow \{0,1\}^\lambda$	$m^* := 0^\lambda$
return k	for $i = 1$ to ℓ:
	$\quad m^* := m^* \oplus m_i$
	return $F(k, m^*)$

Show that the scheme is **not** a secure MAC. Describe a distinguisher and compute its advantage.

A secure MAC does not allow an attacker, who has seen a valid message-tag pair, to forge a valid tag for a different message. Assume you have a valid (m,t) pair. Denote with (m_1,...,m_l) the lambda bit long blocks of the message. Draw a lambda bits string at your leisure, call it pad.

The pair ((m_1^pad,m_2^pad,...,m_l), t) is a valid message-tag pair, where the message differs from m according to the pad you chose.

Practical (Basic) Crypto Challenges

http://overthewire.org/wargames/krypton/

The End